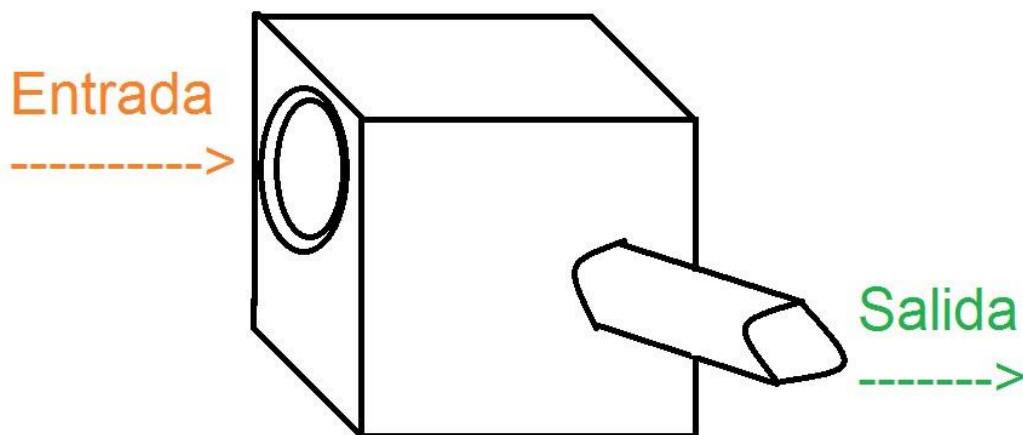


Teórico - Métodos de la clase.

¿Qué es un método?

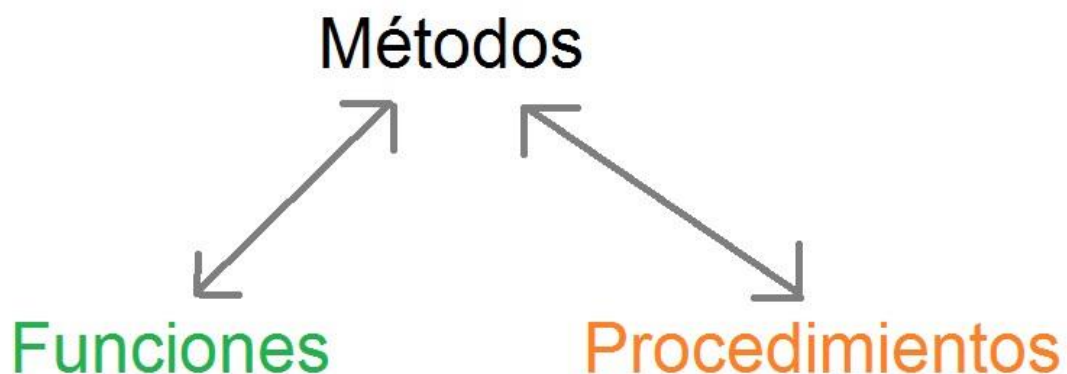
En programación, un método es una colección de instrucciones que realizan una tarea específica y se pueden llamar en cualquier momento durante la ejecución del programa. Se utilizan para dividir el código en tareas más pequeñas y manejables, lo que hace que el código sea más fácil de entender, mantener y reutilizar.

Hay diferentes tipos de métodos, podemos hacer una analogía diciendo que un método es una máquina para llevarlo a algo más tangible. ¿De qué tipo será? Esto dependerá para que lo necesitemos. El clásico ejemplo es una máquina:



A partir de una entrada de datos, la máquina o método los procesa y hace algo con ellos para luego devolver otro dato. En programación podemos encontrarnos con máquinas que tienen N entradas de datos (es decir varias), o ninguna entrada. Y a su vez, respecto a la salida, suelen tener una sola salida de datos, o puede que ninguna.

En programación las Funciones son los métodos que siempre devuelven un dato, el cual posteriormente puede ser usado para algún otro fin dentro del programa. Y si el método no devuelve nada (es decir, devuelve vacío "void") se dice que este es un Procedimiento, el cual puede imprimir algo en pantalla o hacer alguna otra cosa, pero nunca devolver un valor, sino, sería una función.



En síntesis, los métodos serán, o procedimientos o, funciones. Y las funciones o procedimientos serán métodos.

Es importante diferenciar cuando es necesario hacer uno u otro, si un método debe “devolver” o “retornar” significa que debe haber una salida de datos, por ende ser una función. En caso que el método mencione “imprimir” o “mostrar en pantalla” o hacer algo que no sea devolver un dato, estaremos hablando de un procedimiento.

Implementación en Java:

Los métodos de la clase se caracterizan por no necesitar de una instancia para poder ser llamados y utilizados. Algo que se los caracteriza es llevar la palabra “static” en el cabezal al momento de definirlo.

Cada método debe de llevar un nombre y ser declarado en alguna parte del código previamente a hacer uso del mismo. ¿Dónde se declara? Puede hacerse en la misma clase o en otra diferente. Veremos los 2 casos.

Para declararla en el mismo archivo debemos hacerlo, o antes, o después del método main. A continuación se ve marcado en verde:

```
1 package prueba;
2
3 public class principal {
4     
5
6
7     public static void main(String[] args) {
8
9     }
10    
11
12 }
13
14
```

En el medio de los cuadrados es donde posteriormente a la declaración del método se hará uso de la misma solo con su nombre y los parámetros necesarios (método principal).

La otra forma de hacerlo es declarando los métodos en otro archivo diferente, hay que crear otra clase (preferentemente en el mismo paquete, y que no sea una clase main). Luego dentro de la clase podremos declarar las funciones:

```
package prueba;

public class Metodos {
    
}

```

Un punto importante para hacerlo de esta manera, es tener en cuenta el nombre utilizado para el archivo, en este caso “Metodos”, ya que luego desde el archivo principal deberemos no solo poner el nombre del método a utilizar sino antes un indicador para saber de qué archivo se accede al método, por ejemplo, si el método se llama “ejemplo” y el archivo “Metodos”, desde el main para utilizarlo se deberá poner:

```
Metodos.ejemplo();
```

¿Cómo declararlo? ¿Cuáles son sus partes?

Estructura general del método y sus elementos:

```
static tipo_de_dato nombre_del_metodo (param1,param2,paramN) {

    //
    //cuerpo del método
    //

    //salida (de ser una función)
}
```

Al ser una estructura, es necesario respetar el orden de los elementos:

Primero “static” hace referencia a que no instanciaremos una clase para utilizarlo, en esta unidad temática todos los métodos harán uso de esta palabra.

El “tipo_de_dato” hace referencia a un tipo de dato de los que conocemos de programación, y al sustituirlo por uno de los mismos, indicaremos lo que estará devolviendo/retornando el método, si no devuelve nada, se sustituirá por “void”, indicando que es un procedimiento. Otros posibles datos a devolver son “int”, “boolean”, “float”, “char”, etc.

El “nombre_del_metodo” se sustituirá por el nombre que queremos darle al método, el cual servirá luego para cuando queramos utilizarlo, este nombre debería hacer alusión a lo que hace el método.

Luego entre paréntesis y separados por coma (de haber más de uno) deberán ir los parámetros, es decir, la entrada de datos, lo que necesita la máquina para que funcione correctamente. Estos parámetros estarán agrupados por el tipo de dato del parámetro y su nombre. Por ejemplo (int numero, int otroNumero, char letra).

Finalmente entre los paréntesis del método se encuentra el cuerpo del mismo, el cual deberá hacer uso de la entrada ingresada (los parámetros mencionados anteriormente) si la hay, para así resolver el problema de programación para el que se diseña el mismo método. Cabe destacar que ese cuerpo, si el método fuese una función, tendrá que indicar que es lo que devuelve o retorna, y para eso utilizará la palabra reservada “return”, seguida de lo que devolverá.

A continuación dos ejemplos, uno de un procedimiento y otro de una función:

Procedimiento que dado un número entero, imprima que ocurrió un error con ese número:

```
static void msjError(int x) {  
    System.out.println("Ocurrió el error numero "+x);  
}
```

Observese que no devuelve nada, imprime en pantalla, por eso “void”, y también el uso del parametro ingresado “x” en el cuerpo del procedimiento.

Función que dada la base y la altura de un rectángulo, calcule y devuelva el perímetro del mismo como resultado (utilizando enteros):

```
static int perimetroRectangulo(int base, int altura) {  
    int resultado;  
    resultado=(base*2)+(altura*2);  
    return resultado;  
}
```

“int” indica que devolverá un entero, en el cuerpo de la función se ve cómo utiliza “base” y “altura”, los cuales son los parámetros necesarios para poder operar. Se ve como utiliza una variable “resultado” para calcular el perímetro, y al final, devuelve/retorna ese valor, el cual puede guardarse en algún otro lugar.

Invocación:

Una vez declaradas/creadas los métodos, estos desde el programa principal pueden ser usados/invocados siempre que sea necesario. Se verá un ejemplo del método principal invocando a los métodos del ejemplo:

```
public static void main(String[] args) {  
  
    msjError(123);  
  
    int x=perimetroRectangulo(2, 3);  
  
    System.out.println(x);  
}
```

Imprime en pantalla:

```
Ocurrio el error numero 123  
10
```

Puede observarse que solo se necesita poner el nombre del método, en el caso del método "msjError" el parámetro ingresado es "123", podría ser otro número, cualquier entero.

Luego se utiliza una variable "x" para guardar el valor devuelto por la función "perimetroRectangulo" en la cual se ingresan los parámetros 2 y 3. Este es un ejemplo del uso de una función aprovechando el retorno para hacer alguna otra acción, como en este caso guardar e imprimir.

También a los métodos podemos ingresarle variables u operaciones en sus parámetros de entrada y funcionaran correctamente siempre que los datos sean válidos:

```
public static void main(String[] args) {  
    int numero=5;  
    msjError(numero);  
    int x=perimetroRectangulo(numero, numero*2);  
    System.out.println(x);  
}
```

Imprimirá:

```
Ocurrio el error numero 5  
30
```

Se adjunta imagen del código completo del final, el cual tiene no solo la invocación, sino la declaración de los métodos. En rojo el main, y en verde los métodos:

```
package prueba;
```

```
public class principal {
```

```
    static int perimetroRectangulo(int base, int altura) {  
        int resultado;  
        resultado=(base*2)+(altura*2);  
        return resultado;  
    }
```

```
    static void msjError(int x) {  
        System.out.println("Ocurrio el error numero "+x);  
    }
```

```
    public static void main(String[] args) {  
        int numero=5;  
        msjError(numero);  
        int x=perimetroRectangulo(numero, numero*2);  
        System.out.println(x);  
    }
```

```
}
```